Asymptotic Analysis and Time Complexity



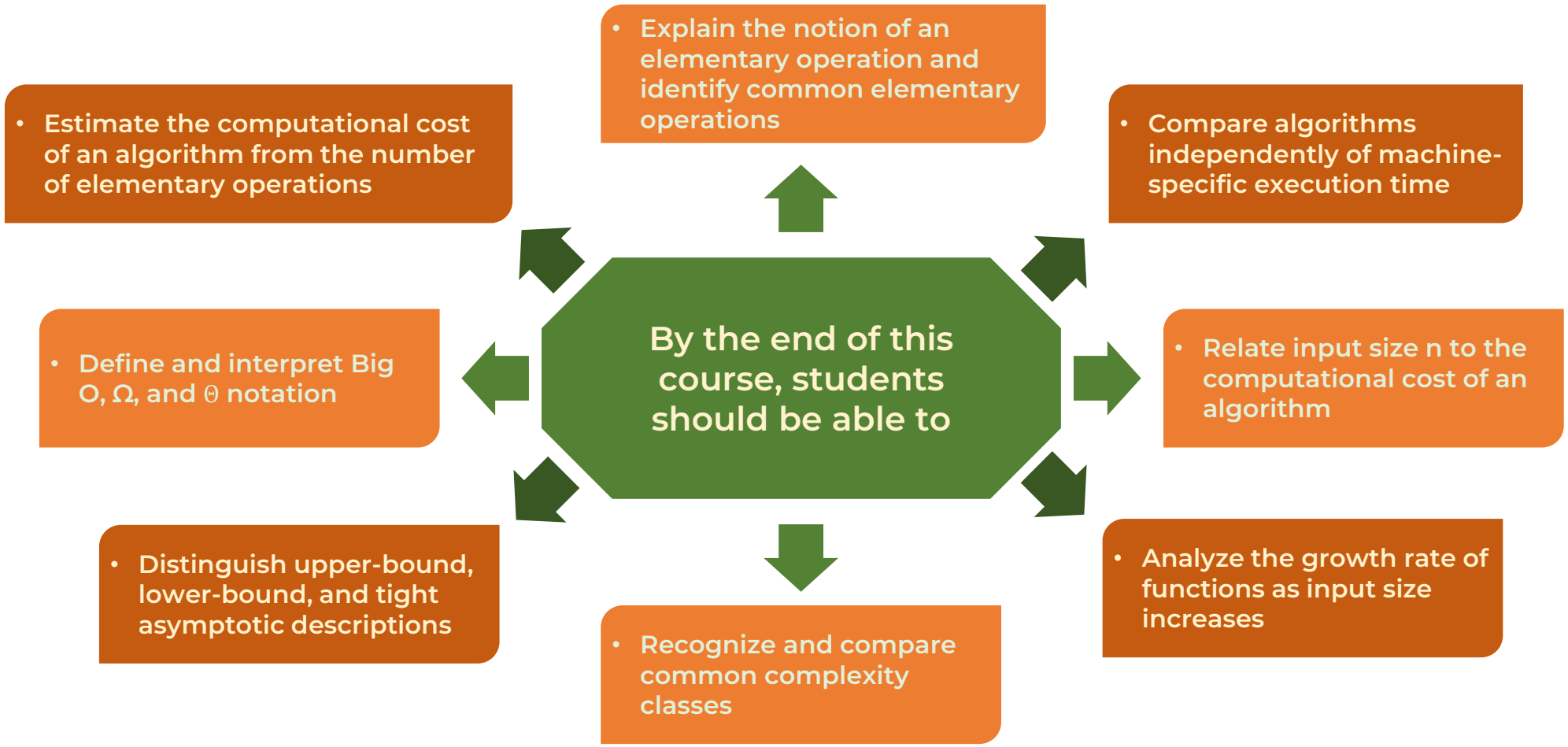
Lecture 2 — Analysis of Time Complexity

Abdelkrim Mebarki
Department of Computer Science

University of Science and Technology of Oran – Mohamed Boudiaf (USTOMB)
Academic Year : 2026 - 2027

https://sites.google.com/view/a-mebarki/

Learning Objectives



Course Roadmap

From operations to growth

1. Elementary operations
2. Computational cost
3. Asymptotic notation
4. Complexity classes
5. Growth-rate comparison
6. Input size and **scalability**

Central question

If two algorithms solve the same problem, how can we describe which one performs fewer computational steps as ***n*** becomes large?

The answer requires a **machine-independent** way to describe growth.

1. The Notion of an Elementary Operation

- An **elementary operation** is a basic computational step that is treated as having a constant cost in the chosen computational model

→ Typical examples:

→Comparison: `x < y` | Assignment: `x ← y`

→Arithmetic operation: `x + y` | Data access: `A[i]`

→Logical operation: `p AND q`

- The exact model may vary, but the goal is the same: **count basic steps** and study how their number changes with input size.

Identifying Elementary Operations

Comparisons and assignments

Comparison → `if A[i] > max`

Assignment → `max ← A[i]`

Comparisons often control loops and branches.

Assignments update variables or data structures.

Arithmetic and data access

Arithmetic → `x + y, 2*i, n/2`

Data access → `A[i], matrix[i][j]`

Different hardware may implement these at different speeds; asymptotic analysis abstracts from those details.

The Computational Model

- To analyze an algorithm, we choose a **simplified model of computation**.
- A common introductory model assumes:
  - →Each **elementary operation** has **constant cost**.
  - →We **count operations** rather than seconds.
  - →**Memory access** is treated **abstractly** unless the model states otherwise.
  - →The **input size** is represented by a parameter such as ***n***.
- This model is not a stopwatch.
  - →It is a mathematical abstraction designed to reveal how **computational work grows**.

Example — Counting Operations

```
sum ← 0
for i ← 1 to n do
    sum ← sum + A[i]
return sum
```

- **What grows with n?**
  1. The loop executes n times.
  2. Each iteration performs a data access, an addition, and an assignment.
  3. A simple count is therefore proportional to n.
  4. The exact constant depends on the chosen cost model.

Measuring the Computational Cost

- Let **C(n)** denote the number of elementary operations performed for an input of size **n**.
- For a given algorithm:
  → **C(n)** depends on the algorithm's control flow.
  → **C(n)** depends on the input size.
  → It may also depend on the **particular input instance**.
  → We can study **best-case**, **worst-case**, or **average/expected** cost.
- The main objective is not usually to obtain an exact number for every machine. We want to understand **the growth of C(n) as n increases**.

Exact Cost vs. Asymptotic Cost

Exact operation count

May look like: **C(n) = 3n + 2**

→ Useful when the computational model is fixed and the **details matter**.

→ It retains **constant factors** and **lower-order** terms.

Asymptotic description

Focuses on dominant growth:

→ C(n) = 3n + 2

→ **grows linearly**

→ We write **O(n)**, **Ω(n)**, or **Θ(n)** depending on the statement being made.

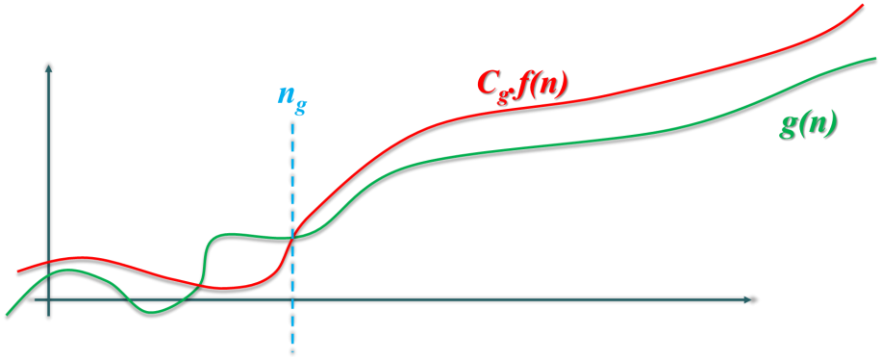
→ **Constants** and **lower-order terms** become less important as n becomes large.

2 / BIG O, Ω, AND Θ

Three standard ways to describe asymptotic growth

Big O Notation — O

- **Big O** provides an asymptotic upper bound.
- For functions f(n) and g(n), we say:
  → **f(n) = O(g(n))**
  → if there exist constants c > 0 and n_0 such that
  → **$0 \leq f(n) \leq c \cdot g(n)$** for all **$n \geq n_0$**
- Interpretation:
  → After some sufficiently large input size,
  → f(n) **does not grow faster than** a constant multiple of g(n).



Mebarki informatics

Interpreting Big O

Think: “at most this growth order”

- Big O is an upper-bound statement.
- Example: **3n + 5 = O(n)**
- Because
  - for sufficiently large n, **3n + 5 ≤ 4n**.
- **Big O** does not mean that the running time **is exactly n operations**.

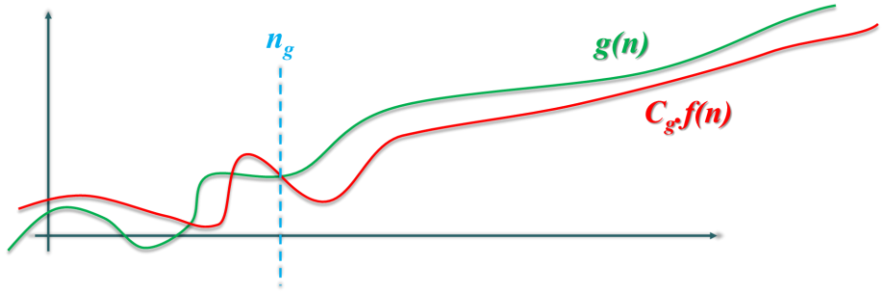
Important

→ An algorithm with **cost n** can also be described as **O(n^2)**, because **n** is eventually bounded above by a constant multiple of **n^2**.

→ Therefore, **O notation** alone does not express the tightest possible bound.

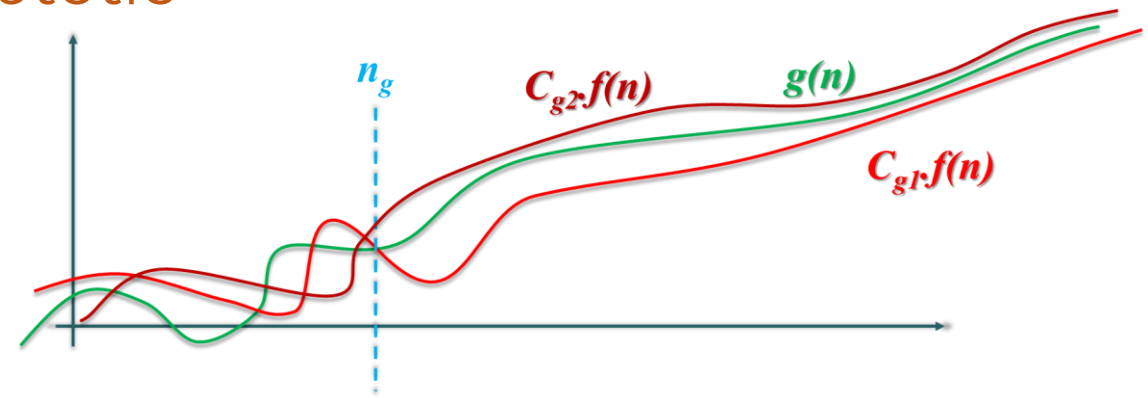
Omega Notation — Ω

- **Omega** provides an **asymptotic lower bound**.
  → **f(n) = Ω(g(n))**
  → if there exist constants c > 0 and n_0 such that
  → **$0 \leq c \cdot g(n) \leq f(n)$** for all **$n \geq n_0$**.
- Interpretation:
  → For sufficiently large **n**, **f(n)** grows at least as fast as a constant multiple of **g(n)**.
- Example:
  → **3n + 5 = Ω(n)**.



Theta Notation — Θ

- **Theta** gives a **tight asymptotic bound**.
  → **f(n) = Θ(g(n))**
  → if there exist positive constants **c_1**, **c_2** and **n_0** such that
  → **$0 \le c_1 g(n) \le f(n) \le c_2 g(n)$** for all **$n \ge n_0$**.
- Interpretation:
  → **f(n)** and **g(n)** grow at the same asymptotic order.
- Example:
  → **$3n + 5 = \Theta(n)$**.



Mebarki informatics

O vs. Ω vs. Θ

Upper and lower bounds

O(g(n))

→ asymptotic upper bound

Ω(g(n))

→ asymptotic lower bound

These bounds answer different questions about growth.

Tight bound

Θ(g(n))

→ Both an upper and a lower bound of the same order.

If **f(n) = Θ(g(n))**, then **f(n)** is asymptotically sandwiched between constant multiples of **g(n)**.

Relationship Between O, Ω, and Θ

- If **f(n) = Θ(g(n))**, then both of the following are true:
  → **f(n) = O(g(n))**
  → **f(n) = Ω(g(n))**
- Conversely, if both **f(n) = O(g(n))** And **f(n) = Ω(g(n))** hold
  → then **f(n) = Θ(g(n))**.
- Thus
  → **Θ** expresses a tight asymptotic characterization,
  →while **O** and **Ω** can be used independently as one-sided bounds.

Mebarki informatics

Worked Example — Polynomial Cost

$$C(n) = 4n^2 + 7n + 3$$

Dominant term

As **n** becomes large, **n^2** grows faster than **n** and **1**.

Therefore: **$C(n) = \Theta(n^2)$**

Consequently:

$C(n) = O(n^2)$

$C(n) = \Omega(n^2)$

The coefficients **4**, **7**, and **3** do not change the asymptotic class.

Asymptotic Complexity

- **Asymptotic complexity** describes the growth of computational cost as the input size becomes large.
- The emphasis is on:
  → The **dominant** growth term.
  → The **relationship** between **n** and computational work.
  → **Scalability** across increasingly large inputs.
  → **Machine-independent** comparison.
- **Examples**:
  → **$\Theta(1)$, $\Theta(\log n)$, $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^2)$, $\Theta(n^3)$, $\Theta(2^n)$, $\Theta(n!)$.**

3 / COMPLEXITY CLASSES

Three standard ways to describe asymptotic growth

O(1) : Constant

- A constant-time algorithm performs

  →**A bounded amount of work that does not depend on n**.
- Example: **return A[0]**
- The number of elementary operations remains approximately constant as n grows.
- Typical situations:

  → Accessing an array element by index.

  → Assigning a fixed number of variables.

  → Performing a fixed number of arithmetic operations.
- **O(1)** describes a growth class, not literally one machine instruction.

O(log n): Logarithmic

- **Logarithmic growth** occurs when each step **reduces the remaining problem size by a constant factor**.
- Typical pattern: **n → n/2 → n/4 → n/8 → …**
- After **k** steps:

  →**$n / 2^k \approx 1$** so **$k \approx \log_2 n$**.
- Classic example: Binary search on a sorted array.
- The base of the logarithm changes only the constant factor,

  → So logarithms with different constant bases belong to the same asymptotic class.

O(n): Linear

- **Linear growth** means computational work is **proportional to n**.
- Example: A single loop that visits each element once.

```
for i ← 1 to n
    process(A[i])
```

- If n doubles, the dominant amount of work also roughly doubles.
- Typical examples:
  → Linear search
  → Summing an array
  → Finding a maximum by scanning all elements

O(n log n): Linearithmic

- **n log n** grows
  → **faster than linear**
  →but much **slower than quadratic**.
- It commonly appears when:
  → **n** items are processed across logarithmically many levels.
  → A divide-and-conquer algorithm performs linear work at each level.
- Classic examples include efficient comparison-based sorting algorithms such as merge sort.
- For large **n**:
  → **$n < n \log n < n^2$**

Mebarki informatics

O(n²): Quadratic

- Often appears when every item is **compared with many other** items.
- Example:

```
for i ← 1 to n
    for j ← 1 to n
        process(i, j)
```

- The nested loops execute approximately **n²** iterations.
- If **n** doubles: **(n)²** becomes **(2n)²** = **4n²**.
- Typical examples:
  → Comparing all ordered pairs
  → Simple quadratic-time sorting algorithms in their worst cases

$O(n^3)$: Cubic

- **Cubic** growth commonly appears with **three nested loops** over **n** items.

```
for i ← 1 to n
    for j ← 1 to n
        for k ← 1 to n
            process(i, j, k)
```

- The number of iterations is n^3.
- Doubling **n** multiplies the dominant work by **8**:
  → $(2n)^3 = 8n^3$.
- **Cubic** algorithms can **become expensive rapidly** as input size increases.

$O(2^n)$: Exponential

- **Exponential** growth occurs when the amount of work multiplies by a constant factor as **n** increases by one.
  → 2^n values:
  → **n** = **10** → **1,024**
  → **n** = **20** → **1,048,576**
  → **n** = **30** → **1,073,741,824**
- Exponential algorithms can **become impractical quickly**.
- They often arise in
  → Exhaustive search over **subsets** or in **recursive** algorithms that branch into multiple possibilities.

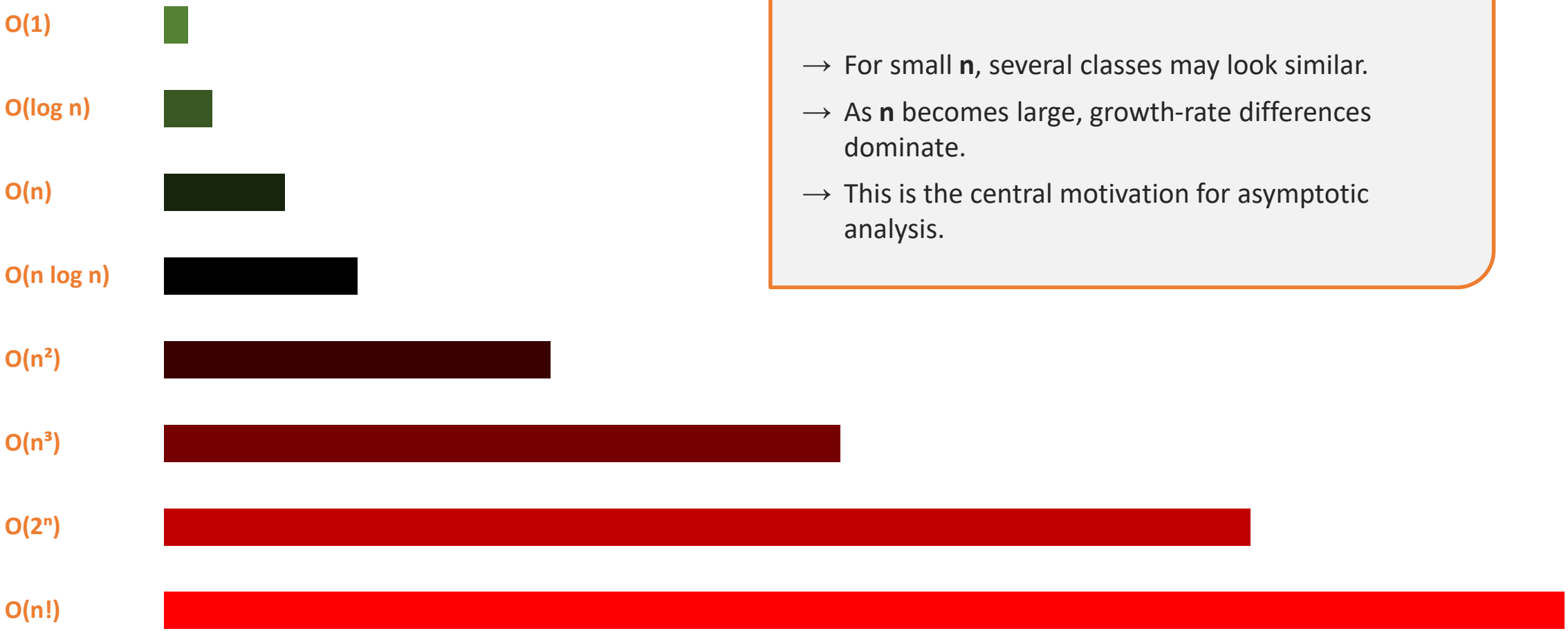
O(n!): Factorial

- **Factorial** growth is even faster than exponential growth.
  → **$n! = n \times (n-1) \times \ldots \times 2 \times 1$**
- Examples:
  → **5!** = **120**
  → **10!** = **3,628,800**
  → **15!** = **1,307,674,368,000**
- **Factorial complexity** often appears when an algorithm explores all **permutations of n objects**.
- For sufficiently large **n**, **n!** dominates **2^n** and the other classes introduced here.

Comparative View

Class	Growth	Typical interpretation
O(1)	Constant	Work stays bounded
O(log n)	Logarithmic	Problem shrinks by a factor
O(n)	Linear	One pass through n items
O(n log n)	Linearithmic	Linear work across log levels
$O(n^2)$	Quadratic	Pairs / two nested loops
$O(n^3)$	Cubic	Triples / three nested loops
$O(2^n)$	Exponential	Subset-style exhaustive growth
O(n!)	Factorial	Permutation-style exhaustive growth

Growth Rates



Key observation (— Why the Difference Matters)

→ For small **n**, several classes may look similar.

→ As **n** becomes large, growth-rate differences dominate.

→ This is the central motivation for asymptotic analysis.

Asymptotic Comparison of Functions

- To compare two functions **f(n)** and **g(n)**
  → Ask which one grows faster **as n becomes large**.
- A useful hierarchy is:
  → **$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n!$**
- The symbol $\prec$ here means
  → "**grows asymptotically more slowly than**."
- This hierarchy is a conceptual guide.
  → Exact comparisons can be established mathematically using **limits**, **inequalities**, or **known asymptotic rules**.

Dominant Terms and Simplification

- When analyzing a polynomial:
  → **$f(n) = 6n^3 + 20n^2 + 100n + 7$**
  → the **n^3** term dominates as **n becomes large**.
- Therefore:
  → **$f(n) = \Theta(n^3)$**
- **General principle**:
  → Ignore **constant multiplicative factors**.
  → Ignore **lower-order terms**.
  → Keep the **fastest-growing term**.
- This simplification is what makes asymptotic classes useful for comparing algorithms.

Input Size & Computational Cost

- Let **n** represent the relevant input size.
- Then an algorithm can be described by a cost function **C(n)**.
- Examples:
  - → Array of **n** elements → **n** is the number of elements.
  - → String of **n** characters → **n** is the string length.
  - → Graph → size may require both **|V|** and **|E|**.
  - → Matrix → dimensions **m × n** may be more appropriate.
- **Choosing the right size** parameter is essential: the complexity statement is meaningful only when **n** correctly captures the amount of input being processed.

Same Input Size, Different Work

Algorithm A

→ A single scan:

```
for i = 1 … n
    process(A[i])
```

→ Approximate growth: **Θ(n)**

→ If **n** doubles, work roughly doubles.

Algorithm B

→ All pairs:

```
for i = 1 … n
    for j = 1 … n
        compare(A[i], A[j])
```

→ Approximate growth: **Θ(n²)**

→ If **n** doubles, work roughly quadruples.

4 / WORKED EXAMPLES

Three standard ways to describe asymptotic growth

Why Binary Search Is Logarithmic

- Suppose a sorted array contains **n** elements.
- Binary search repeatedly **halves** the remaining interval:
  → **$n \to n/2 \to n/4 \to \ldots \to n/2^k$**
  → Stop when: **$n/2^k \leq 1$**
- This gives: **$2^k \geq n$**
- and therefore: **$k \geq \log_2 n$**
- So the number of comparisons is
  → **$\Theta(\log n)$** in the worst case.
- The logarithmic behavior comes directly from repeatedly discarding half of the remaining search space.

Mebarki informatics

Nested Loops

```
for i ← 1 to n do
    for j ← 1 to n do
        x ← x + 1
```

Counting the iterations

- The outer loop executes **n** times.
- For each outer iteration, **the inner loop executes n times**.
- Total iterations = **$n \times n = n^2$**.
- Therefore the running time is **$\Theta(n^2)$**.

Common Misunderstandings

Big O is not an exact runtime

→ **O(n)** does not mean **“n seconds”** or exactly **n operations**.

→ It describes an asymptotic upper-bound class.

Big O is not automatically the tightest bound

→ If **$f(n)=\Theta(n)$**, it is also true that **$f(n)=O(n^2)$**.

→ When we want the exact asymptotic growth order, **Θ** is the appropriate notation.

From Operation Counts to Scalability

- Suppose two algorithms require approximately:
  → Algorithm A: **10n operations |** Algorithm B: **n^2 operations**
- For n = 10:
  → **A ≈ 100**, **B ≈ 100**
- For n = 1,000:
  → **A ≈ 10,000**, **B ≈ 1,000,000**
- For n = 1,000,000:
- **A ≈ 10,000,000**, **B ≈ 1,000,000,000,000**
- The exact constants matter in practice,
  →but asymptotic growth reveals why algorithms that seem similar for small inputs can behave very differently at scale.

What We Have Learned

- Elementary operations provide a basic unit for estimating computational work.
- Computational cost can be represented by a function C(n).
  → Big O describes an asymptotic upper bound.
  → Ω describes an asymptotic lower bound.
  → Θ describes a tight asymptotic bound.
- Asymptotic analysis focuses on growth as n becomes large.
- Common classes include
  → $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, and $O(n!)$.
- Faster-growing functions become increasingly expensive as input size increases.
- The choice of input-size parameter is fundamental to meaningful complexity analysis.

Practice Questions

- 1. Identify the elementary operations in a linear search.
- 2. Estimate the operation count for **summing n values**.
- 3. Explain why **5n + 20** is **$\Theta(n)$**.
- 4. Explain why **$3n^2 + 7n + 4$** is **$\Theta(n^2)$**.
- 5. Distinguish **$O(n)$**, **$\Omega(n)$**, and **$\Theta(n)$**.
- 6. Why is binary search logarithmic? 8. Determine the complexity of two nested loops over **n** elements.
- 9. Give an example of an **$O(1)$** operation and an **$O(n)$** operation.
- 10. Explain how doubling **n** affects **$O(n)$**, **$O(n^2)$**, and **$O(n^3)$**.